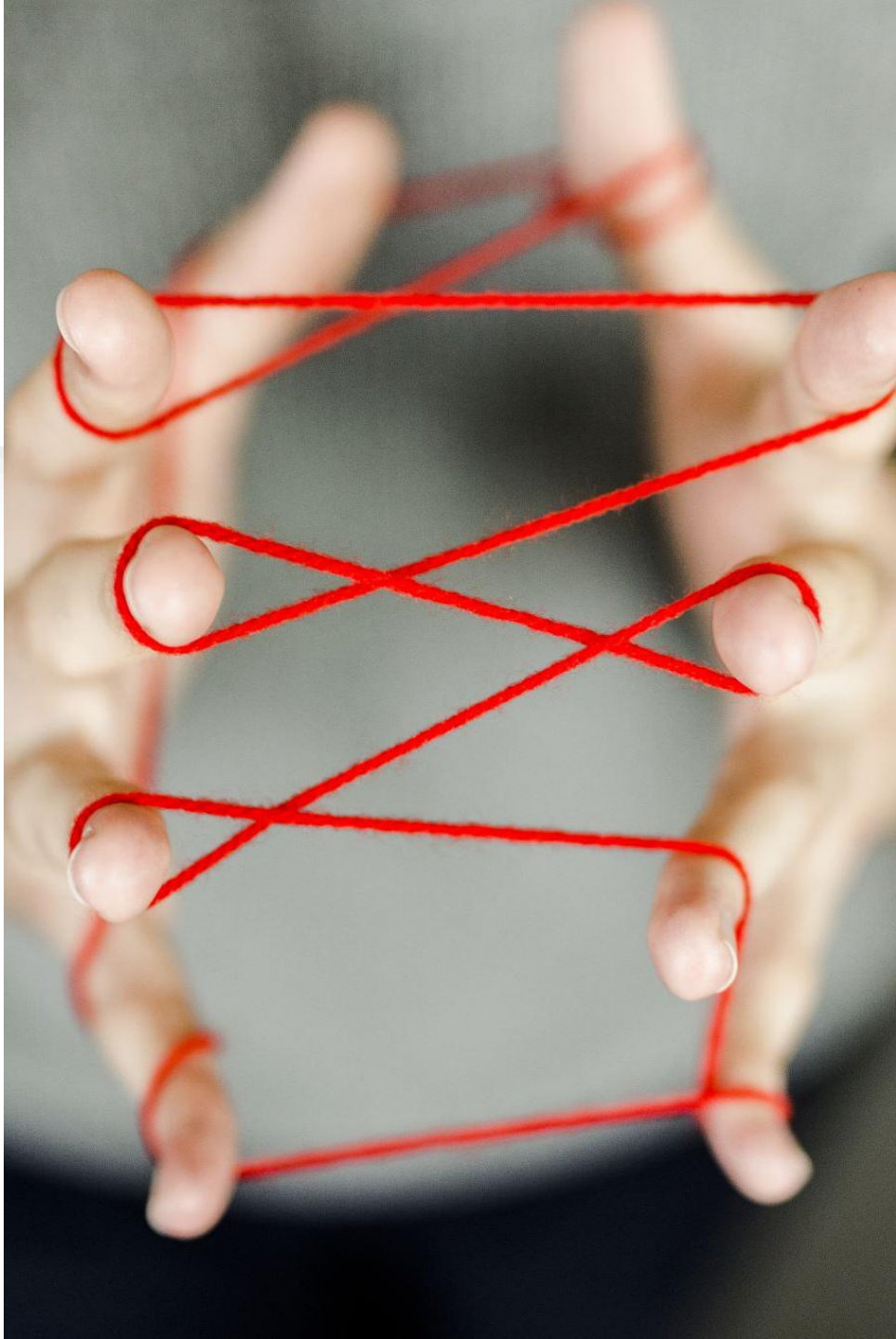
Workshop: Image segmentation

Prof Jamshid Dehmeshki

Task 1: Segmentation

- Load an image
- Simple Thresholding (with all variants)
- Adaptive Thresholding (Mean and Gaussian)
- Otsu's Method (with and without Gaussian Blur)



Welcome | ThresholdingSyn.py

ThresholdingSyn.py > ...

```python
# Thresholding Techniques Tutorial using OpenCV

import cv2
import numpy as np
import matplotlib.pyplot as plt

# ------------------------------------------
# 1. Simple Thresholding
# ------------------------------------------
# Create a synthetic grayscale gradient image
simple_img = np.tile(np.arange(0, 256, dtype=np.uint8), (100, 1))

# Apply different simple thresholding techniques
_, binary = cv2.threshold(simple_img, 127, 255, cv2.THRESH_BINARY)
_, binary_inv = cv2.threshold(simple_img, 127, 255, cv2.THRESH_BINARY_INV)
_, trunc = cv2.threshold(simple_img, 127, 255, cv2.THRESH_TRUNC)
_, tozero = cv2.threshold(simple_img, 127, 255, cv2.THRESH_TOZERO)
_, tozero_inv = cv2.threshold(simple_img, 127, 255, cv2.THRESH_TOZERO_INV)

# Plot results for Simple Thresholding
fig, axes = plt.subplots(2, 3, figsize=(15, 8))
simple_titles = ['Original', 'Binary', 'Binary INV', 'Trunc', 'ToZero', 'ToZero INV']
simple_images = [simple_img, binary, binary_inv, trunc, tozero, tozero_inv]

for i, ax in enumerate(axes.ravel()):
    ax.imshow(simple_images[i], cmap='gray', vmin=0, vmax=255)
    ax.set_title(simple_titles[i])
    ax.axis('off')
plt.suptitle("1. Simple Thresholding", fontsize=16)
plt.tight_layout()
plt.show()
```

```
32
33  # ---------------------------------------------
34  # 2. Adaptive Thresholding
35  # ---------------------------------------------
36  # Create an image with uneven illumination
37  x = np.linspace(0, 255, 256).astype(np.uint8)
38  adaptive_img = np.tile(x, (256, 1))
39  cv2.circle(adaptive_img, (128, 128), 60, (0,), -1)  # Dark circle in center
40
41  # Apply adaptive thresholding
42  adaptive_mean = cv2.adaptiveThreshold(adaptive_img, 255,
43                                        cv2.ADAPTIVE_THRESH_MEAN_C,
44                                        cv2.THRESH_BINARY, 11, 5)
45
46  adaptive_gaussian = cv2.adaptiveThreshold(adaptive_img, 255,
47                                            cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
48                                            cv2.THRESH_BINARY, 11, 5)
49
50  # Plot results for Adaptive Thresholding
51  fig, axes = plt.subplots(1, 3, figsize=(15, 5))
52  adaptive_titles = ['Original (Uneven Lighting)', 'Adaptive Mean', 'Adaptive Gaussian']
53  adaptive_images = [adaptive_img, adaptive_mean, adaptive_gaussian]
54
55  for i, ax in enumerate(axes):
56      ax.imshow(adaptive_images[i], cmap='gray', vmin=0, vmax=255)
57      ax.set_title(adaptive_titles[i])
58      ax.axis('off')
59  plt.suptitle("2. Adaptive Thresholding", fontsize=16)
60  plt.tight_layout()
61  plt.show()
62
```

```python
# ---------------------------------------------
# 3. Otsu's Thresholding
# ---------------------------------------------
# Create bimodal image (two distinct intensity levels)
otsu_img = np.zeros((100, 256), dtype=np.uint8)
otsu_img[:, :128] = 60
otsu_img[:, 128:] = 200

# Add random noise to simulate real-world case
noise = np.random.normal(0, 15, otsu_img.shape).astype(np.uint8)
otsu_img = cv2.add(otsu_img, noise)

# Apply Otsu's thresholding
_, otsu_thresh = cv2.threshold(otsu_img, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)

# Apply Gaussian Blur before Otsu
blurred = cv2.GaussianBlur(otsu_img, (5, 5), 0)
_, otsu_blur_thresh = cv2.threshold(blurred, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)

# Plot results for Otsu's Method
fig, axes = plt.subplots(1, 3, figsize=(15, 5))
otsu_titles = ['Original Bimodal + Noise', "Otsu's Threshold", "Otsu's with Gaussian Blur"]
otsu_images = [otsu_img, otsu_thresh, otsu_blur_thresh]

for i, ax in enumerate(axes):
    ax.imshow(otsu_images[i], cmap='gray', vmin=0, vmax=255)
    ax.set_title(otsu_titles[i])
    ax.axis('off')
plt.suptitle("3. Otsu's Thresholding", fontsize=16)
plt.tight_layout()
plt.show()

```

1. Simple Thresholding

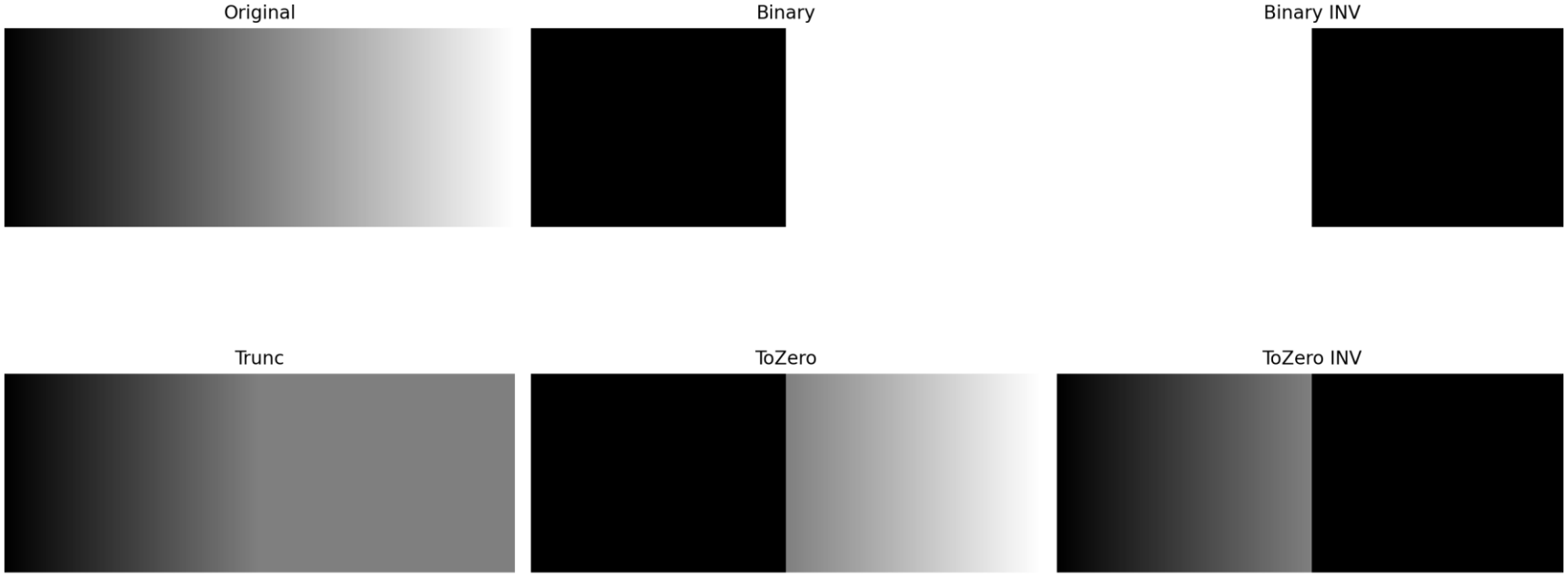
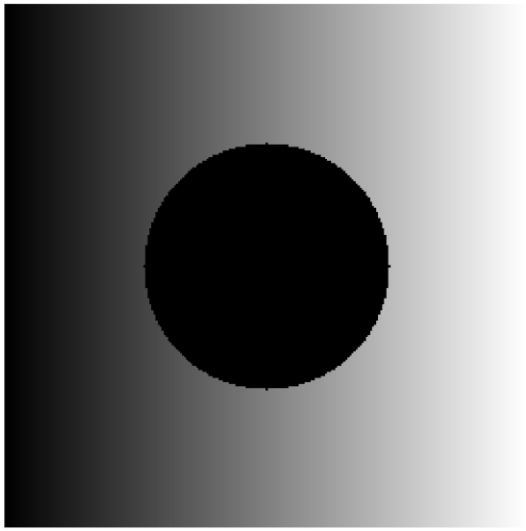


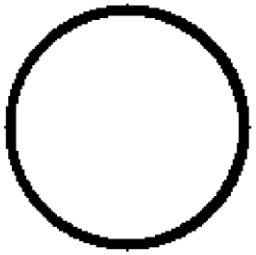
Figure 1

2. Adaptive Thresholding

Original (Uneven Lighting)



Adaptive Mean



Adaptive Gaussian

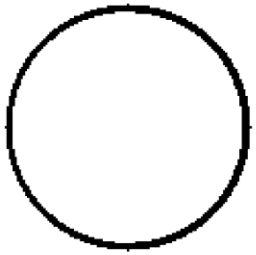
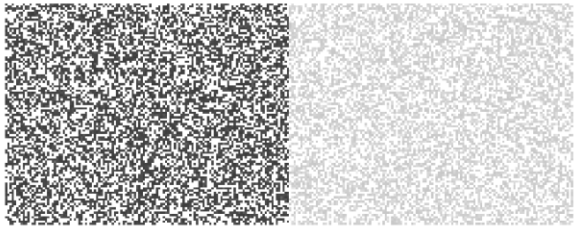


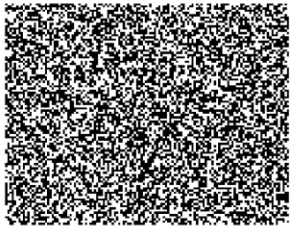
Figure 1

3. Otsu's Thresholding

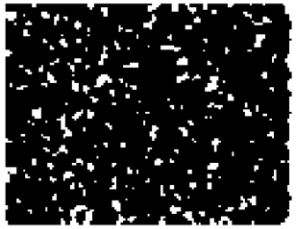
Original Bimodal + Noise



Otsu's Threshold



Otsu's with Gaussian Blur



Assignment 1:

Load one grey level image/convert colour to grey level image

Apply all above exercise to see different results

Task 2: segmentation and object detection

1. Load a grayscale image.
2. Apply Otsu thresholding to convert it into a binary image.
3. Compute connected components (with descriptors such as area, perimeter, bounding box, and centroid).
4. Create a color-labeled image and an annotated image that displays only the label number (with bounding boxes) on each object using the same (original) image size.
5. Display four separate figures (grayscale, binary, color-labeled, annotated) and a separate table that lists each region's measurements.

rgb_to_cmy.py | RegionLabeling.py | distance_measurment.py | LoGFiltering.py | Marr_Hildreth.py | Co

C: > Users > jdehm > OneDrive > Documents > Presentation > Kingston > ComputerVisionCourse2025 > DigitalImageProcessing2025 > Works

```python
import cv2
import numpy as np
import matplotlib.pyplot as plt

def show_measurements_table(measurements):
    """
    Display a table of measurement descriptors (Label, Area, Perimeter, Bounding Box, Centroid).
    """
    # Define column headers
    col_labels = ["Label", "Area", "Perimeter", "Bounding Box", "Centroid"]

    # Prepare table rows
    cell_text = []
    for m in measurements:
        row = [
            m['Label'],
            m['Area'],
            f"{m['Perimeter']:.1f}",
            m['BBox'],
            (f"{m['Centroid'][0]:.2f}", f"{m['Centroid'][1]:.2f}")
        ]
        cell_text.append(row)

    # Create a figure for the table
    fig, ax = plt.subplots(figsize=(8, 4))
    ax.set_axis_off()  # Hide axes
    table = ax.table(cellText=cell_text, colLabels=col_labels, loc='center')

    table.auto_set_font_size(False)
    table.set_fontsize(10)
    table.auto_set_column_width(col=list(range(len(col_labels))))

    plt.title("Object Measurements")
    plt.tight_layout()
    plt.show()

```

```python
def main():
    # 1) LOAD GRAYSCALE IMAGE
    img_gray = cv2.imread('images/Circle.png', cv2.IMREAD_GRAYSCALE)
    if img_gray is None:
        print("Error: Could not load image. Check 'your_image.png' path.")
        return

    # 2) OTSU THRESHOLDING
    _, binary = cv2.threshold(img_gray, 0, 255, cv2.THRESH_BINARY | cv2.THRESH_OTSU)

    # 3) CONNECTED COMPONENTS (8-CONNECTIVITY)
    # This returns: num_labels, labels_im, stats, centroids.
    num_labels, labels_im, stats, centroids = cv2.connectedComponentsWithStats(binary, connectivity=8)

    # Create a color-labeled image for visualization.
    label_hue = np.uint8(179 * labels_im / (num_labels if num_labels > 1 else 1))
    blank_ch = 255 * np.ones_like(label_hue)
    colored_labels = cv2.merge([label_hue, blank_ch, blank_ch])
    colored_labels = cv2.cvtColor(colored_labels, cv2.COLOR_HSV2BGR)
    colored_labels[label_hue == 0] = 0  # Background set to black

    # 4) CALCULATE MEASUREMENTS & CREATE AN ANNOTATED IMAGE (LABEL NUMBERS ONLY)
    measurements = []
    annotated = colored_labels.copy()  # Create a copy for annotation
    for label_id in range(1, num_labels):  # Skip label 0 (background)
        x, y, w, h, area = stats[label_id]
        cx, cy = centroids[label_id]  # Centroid (floating point)

        # Get perimeter using contours.
        region_mask = (labels_im == label_id).astype(np.uint8)
        contours, _ = cv2.findContours(region_mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_NONE)
        perimeter = 0.0
        if contours:
            perimeter = cv2.arcLength(contours[0], closed=True)

```

```
        measurements.append({
            'Label': label_id,
            'Area': area,
            'Perimeter': perimeter,
            'BBox': (x, y, w, h),
            'Centroid': (cx, cy)
        })

        # Draw bounding box in white.
        cv2.rectangle(annotated, (x, y), (x+w, y+h), (255, 255, 255), 2)
        # Put label number in red (using a moderate font size).
        text_str = f"{label_id}"
        text_pos = (x, y - 5)
        cv2.putText(annotated, text_str, text_pos,
                    fontFace=cv2.FONT_HERSHEY_SIMPLEX,
                    fontScale=0.5,
                    color=(0, 0, 255),  # Red color
                    thickness=1,
                    lineType=cv2.LINE_AA)

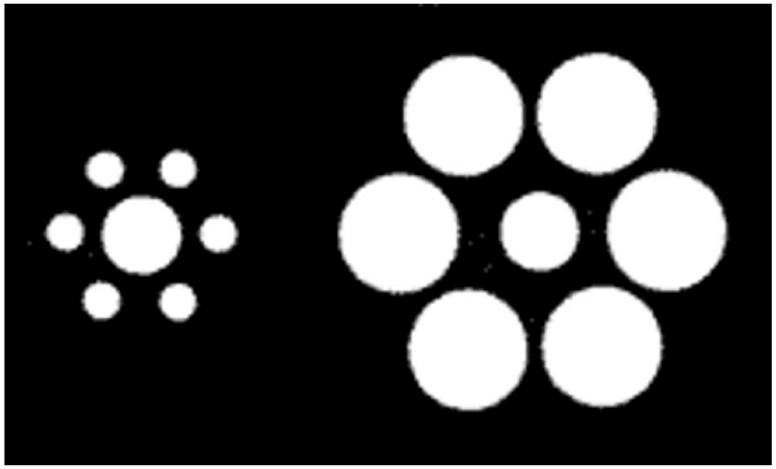
    # Convert BGR images to RGB for proper display in matplotlib.
    colored_rgb = cv2.cvtColor(colored_labels, cv2.COLOR_BGR2RGB)
    annotated_rgb = cv2.cvtColor(annotated, cv2.COLOR_BGR2RGB)

    # 5) DISPLAY THE IMAGES SEPARATELY
    # Original Grayscale Image
    plt.figure(figsize=(6, 6))
    plt.imshow(img_gray, cmap='gray')
    plt.title("1) Original Grayscale Image")
    plt.axis('off')

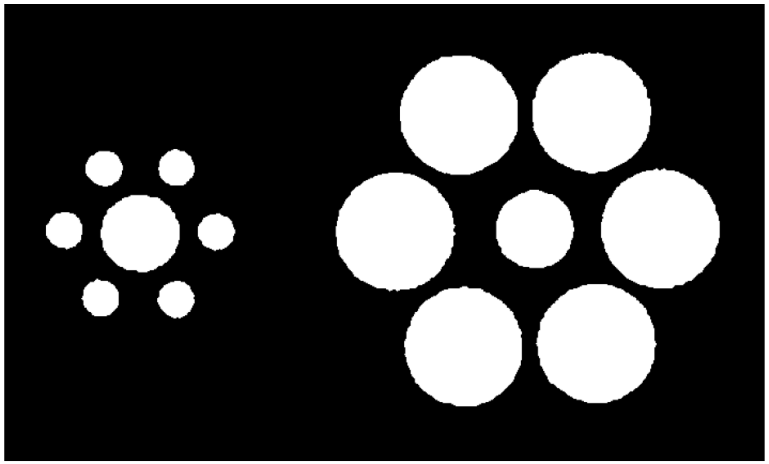
    # Binary Image
    plt.figure(figsize=(6, 6))
    plt.imshow(binary, cmap='gray')
    plt.title("2) Binary Image (Otsu Threshold)")
    plt.axis('off')
```

```
108
109     # Color-Labeled Image (No Annotation)
110     plt.figure(figsize=(6, 6))
111     plt.imshow(colored_rgb)
112     plt.title("3) Color-Labeled Regions")
113     plt.axis('off')
114
115     # Annotated Image (Label Numbers Only, original size)
116     plt.figure(figsize=(6, 6))
117     plt.imshow(annotated_rgb)
118     plt.title("4) Annotated Regions (Label Numbers)")
119     plt.axis('off')
120
121     plt.show()
122
123     # 6) DISPLAY THE MEASUREMENTS TABLE
124     show_measurements_table(measurements)
125
126 if __name__ == "__main__":
127     main()
128
```

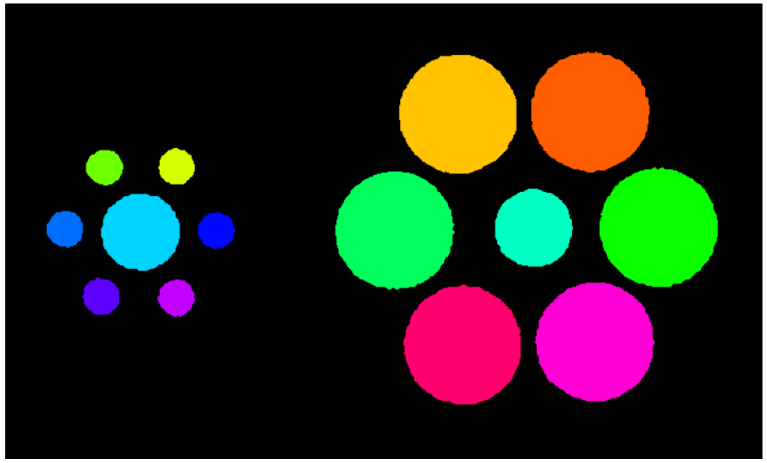
1) Original Grayscale Image



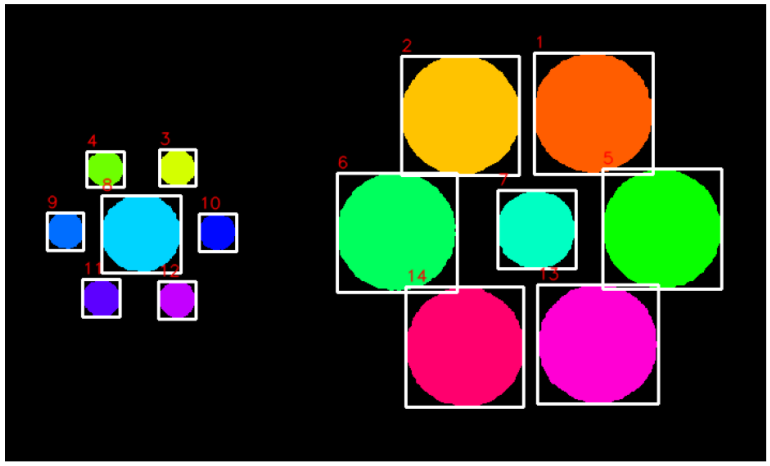
2) Binary Image (Otsu Threshold)

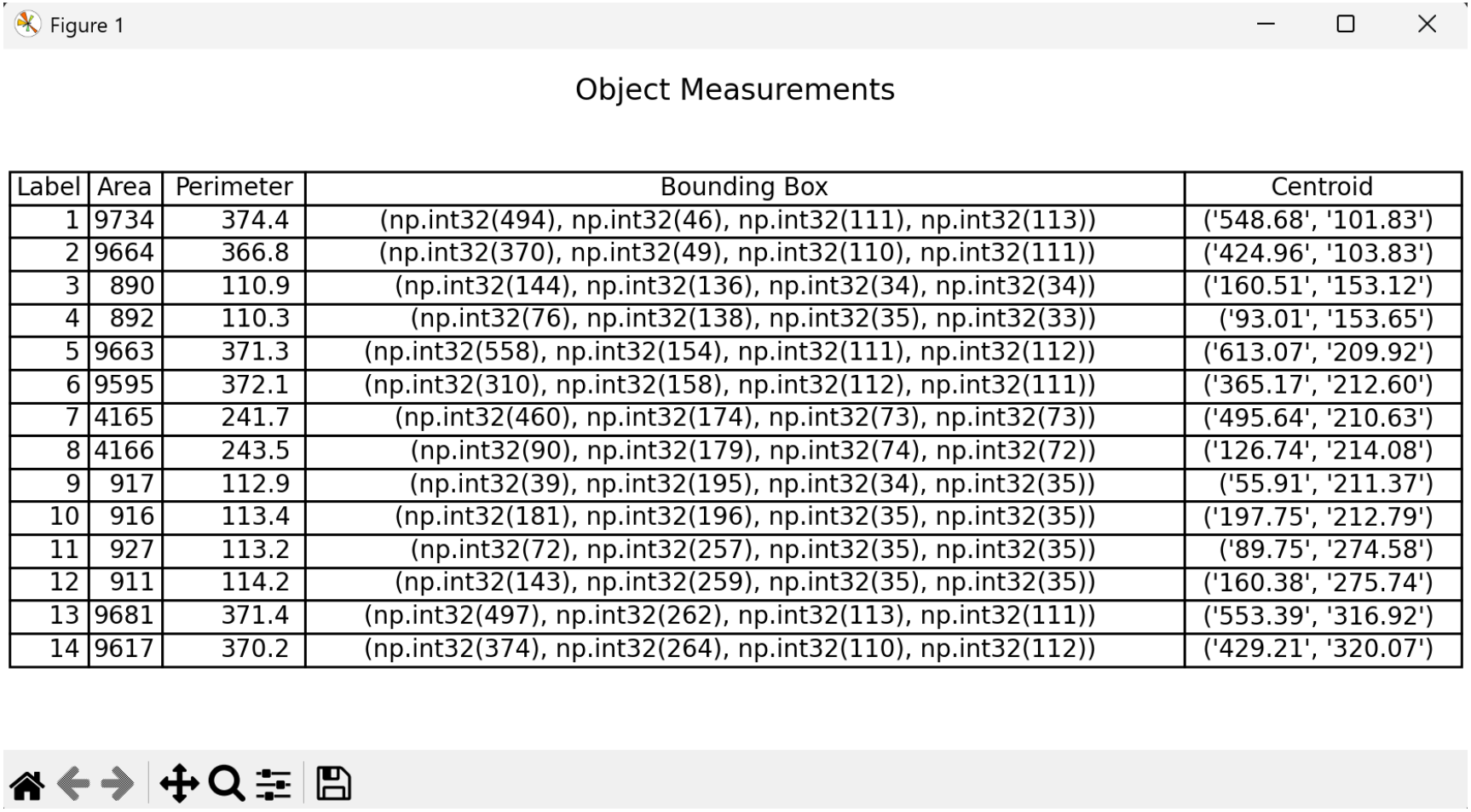


3) Color-Labeled Regions

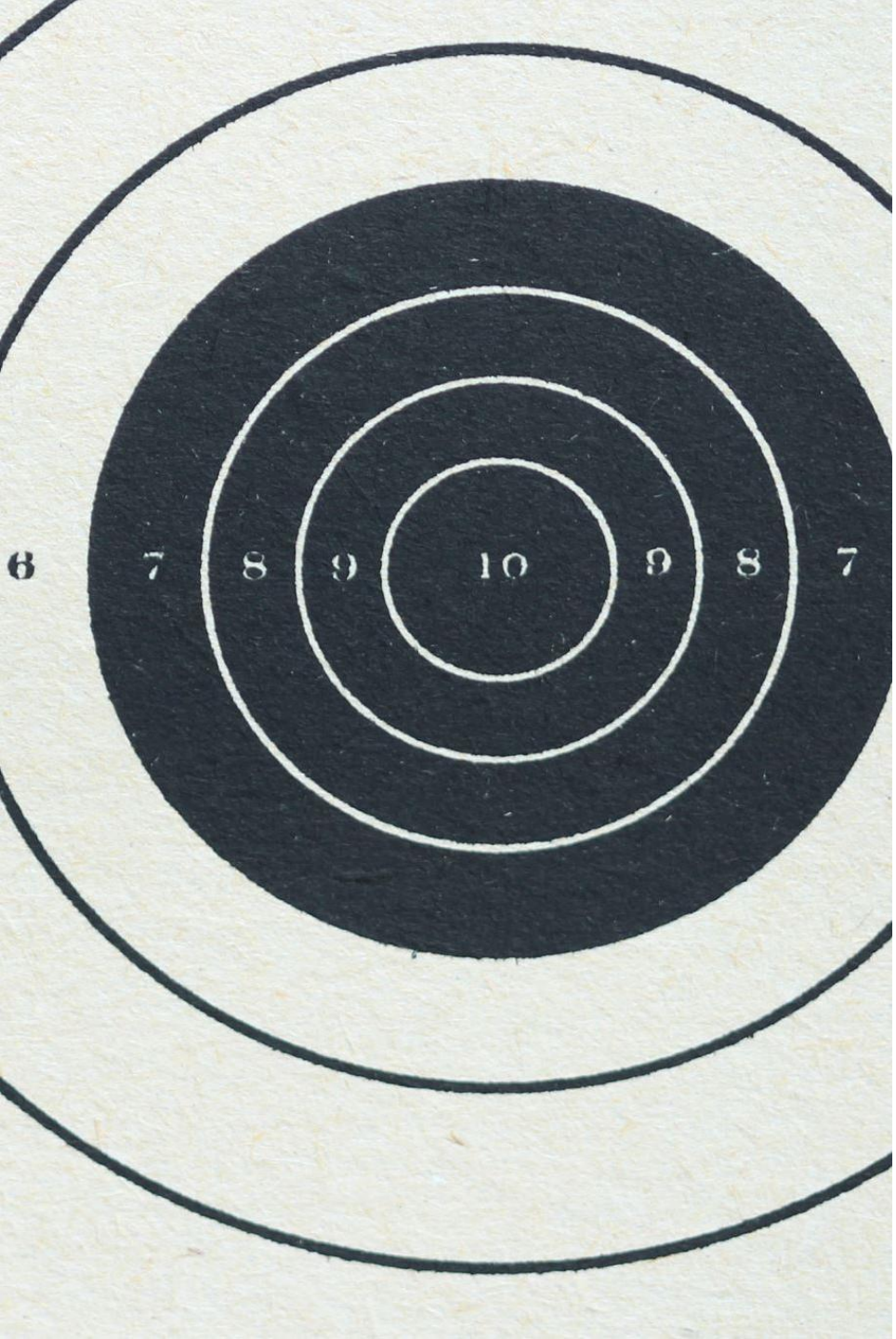


4) Annotated Regions (Label Numbers)





Label	Area	Perimeter	Bounding Box	Centroid
1	9734	374.4	(np.int32(494), np.int32(46), np.int32(111), np.int32(113))	('548.68', '101.83')
2	9664	366.8	(np.int32(370), np.int32(49), np.int32(110), np.int32(111))	('424.96', '103.83')
3	890	110.9	(np.int32(144), np.int32(136), np.int32(34), np.int32(34))	('160.51', '153.12')
4	892	110.3	(np.int32(76), np.int32(138), np.int32(35), np.int32(33))	('93.01', '153.65')
5	9663	371.3	(np.int32(558), np.int32(154), np.int32(111), np.int32(112))	('613.07', '209.92')
6	9595	372.1	(np.int32(310), np.int32(158), np.int32(112), np.int32(111))	('365.17', '212.60')
7	4165	241.7	(np.int32(460), np.int32(174), np.int32(73), np.int32(73))	('495.64', '210.63')
8	4166	243.5	(np.int32(90), np.int32(179), np.int32(74), np.int32(72))	('126.74', '214.08')
9	917	112.9	(np.int32(39), np.int32(195), np.int32(34), np.int32(35))	('55.91', '211.37')
10	916	113.4	(np.int32(181), np.int32(196), np.int32(35), np.int32(35))	('197.75', '212.79')
11	927	113.2	(np.int32(72), np.int32(257), np.int32(35), np.int32(35))	('89.75', '274.58')
12	911	114.2	(np.int32(143), np.int32(259), np.int32(35), np.int32(35))	('160.38', '275.74')
13	9681	371.4	(np.int32(497), np.int32(262), np.int32(113), np.int32(111))	('553.39', '316.92')
14	9617	370.2	(np.int32(374), np.int32(264), np.int32(110), np.int32(112))	('429.21', '320.07')



Assignment 2: Object detection

- Detect largest objects
- Detect nearest object to left side

Detect only two central circles